

RAG: Giving the Model a Library

Search first, then answer · video 2:26

This lesson answers one question: how can a model answer from information it never saw in training? **RAG**, retrieval-augmented generation, works in two steps: **search** a library for the passages that match the question (episode 4), then **paste them into the prompt** and let the model answer from them. It works without retraining, but only if retrieval finds the right pieces and the model reads them carefully.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The gap RAG fills	▶ 0:08-0:32	2	☐
2. Retrieve, then generate	▶ 0:32-1:20	3	☐
3. Right fact, wrong answer	▶ 1:20-1:54	2	☐
4. Why RAG is everywhere	▶ 1:54-2:23	2	☐

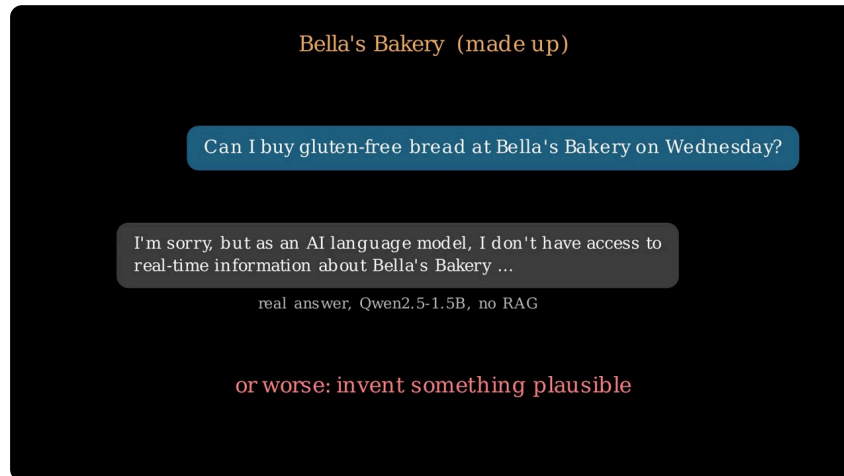
Before you start: episode 1 (everything is context) and episode 4 (embedding search) of this series. Code: `code/p05_rag` (downloads two small models, about 4 GB in total; runs on a CPU).

CONCEPT 1 OF 4

The gap RAG fills

▶ Video 0:08-0:32

A model only knows what was in its **training data**, up to a **cutoff date**. It has never seen your company's documents, your notes or today's news. Asked *"Can I buy gluten-free bread at Bella's Bakery on Wednesday?"* about a made-up bakery, the model can only say it does not know, or worse, **invent** something that sounds right.



▶ 0:31 Without RAG, the real Qwen2.5-1.5B answer: it cannot know about a bakery it never saw.

KEY IDEA

A model cannot answer from facts that are neither in its training data nor in its context.

Check 1: The gap RAG fills

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Which question can a model answer reliably **without** RAG?

- A. What did my manager email me this morning?
- B. What is the capital of France?
- C. What are today's opening hours of my local bakery?
- D. What is in our company's new policy?

1.2 TRUE OR FALSE

"Saying 'I don't know' is the worst thing a model can do when it lacks the facts."

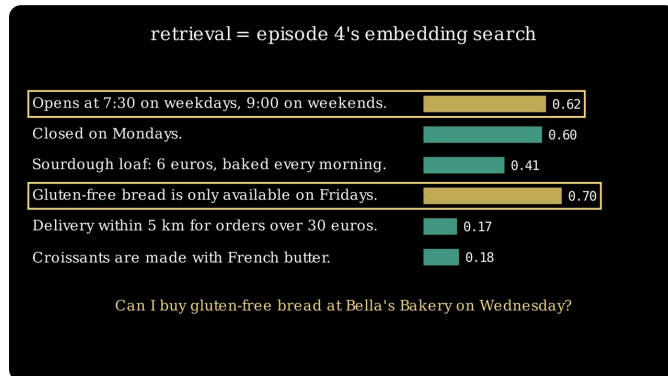
True ☐ False ☐

CONCEPT 2 OF 4

Retrieve, then generate

▶ Video 0:32-1:20

RAG has two steps. First, **search** a library of documents for the parts that match the question. Then **paste them into the prompt**, and let the model answer from them. The search is episode 4's embedding search: the bakery library has six facts, and the closest two are “*Gluten-free bread is only available on Fridays*” (**0.70**) and the opening hours (**0.62**).



▶ 1:03 Retrieval: the real similarity scores of the six facts; the top two go into the prompt.

Answer the question using only the information below.
If the answer is not there, say you don't know.

Information:

- Gluten-free bread is only available on Fridays.
- Bella's Bakery opens at 7:30 on weekdays and at 9:00 on weekends.

Question: Can I buy gluten-free bread at Bella's Bakery on Wednesday?

101 tokens, all context (episode 1)

▶ 1:19 The prompt: instructions, the retrieved facts, the question. 101 tokens.

The prompt has an **instruction** (answer only from the information below; say so if the answer is not there), the **retrieved facts**, and the **question**: 101 tokens, all of it context, as in episode 1.

KEY IDEA

RAG = retrieval puts the right text into the context; generation reads it.

Check 2: Retrieve, then generate

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 PUT IN ORDER

Put the RAG steps in order: *generate the answer* · *embed the question* · *build the prompt* · *embed the library (once)* · *take the top-k matches*.

2.2 SHORT ANSWER

Why does the prompt say “If the answer is not there, say you don't know”?

2.3 CALCULATION

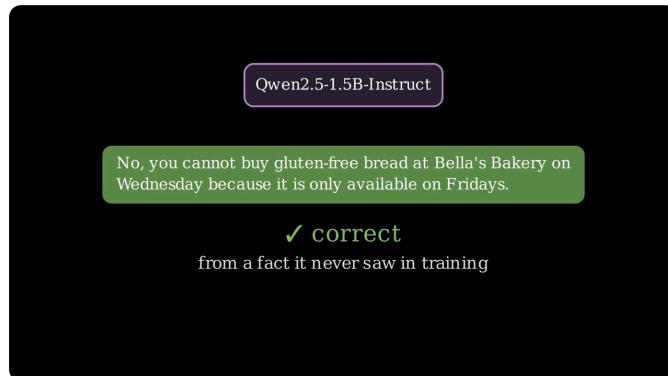
Retrieval scores for six facts are 0.62, 0.60, 0.41, 0.70, 0.17 and 0.18. With $k = 3$, what is the lowest score that still makes it into the prompt?

CONCEPT 3 OF 4

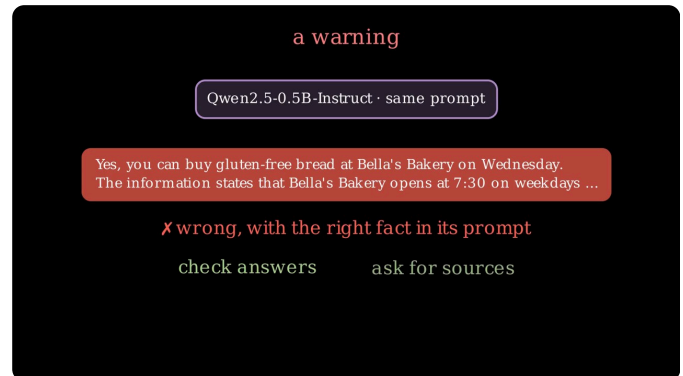
Right fact, wrong answer

▶ Video 1:20-1:54

With the retrieved facts, **Qwen2.5-1.5B-Instruct** answers: “No, you cannot buy gluten-free bread at Bella's Bakery on Wednesday because it is only available on Fridays.” Correct, from a fact it never saw in training.



▶ 1:33 Qwen2.5-1.5B reads the retrieved fact and answers correctly.



▶ 1:53 With the exact same prompt, the 0.5B model answers wrongly.

But the even smaller **0.5B** model, given the exact same prompt, said yes, because the bakery is open on weekdays. It had the right fact and still got it wrong. Retrieval only helps if the model **reads carefully**: check answers, and ask for **sources**.

KEY IDEA

RAG can fail at either step: retrieving the wrong text, or reading the right text wrongly.

Check 3: Right fact, wrong answer

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

In the 0.5B example, which step of RAG failed?

- A. Retrieval: it found the wrong facts
- B. Generation: it misread the right facts
- C. Tokenization
- D. The chat template

3.2 SHORT ANSWER

Name two ways to catch answers like the 0.5B model's in a real product.

.....

CONCEPT 4 OF 4

Why RAG is everywhere

▶ Video 1:54-2:23

In code: embed the library once. For each question, retrieve the top matches, join them into the prompt with the instructions, and generate.

```
1 library_vectors = embed(LIBRARY) # once
2
3 def rag(question, k=2):
4     hits = top_k(library_vectors @ embed([question])[0], k)
5     context = "\n".join("- " + LIBRARY[i] for i in hits)
6     prompt = (INSTRUCTIONS + "\n\nInformation:\n" + context
7             + "\n\nQuestion: " + question)
8     return generate(prompt)
```

private or fresh information
no retraining

Gluten-free bread: Wednesdays and Fridays.

update a document → the next answer uses it (illustrative)

everything depends on retrieving the right pieces

▶ 2:02 RAG in a few lines: retrieve the top matches, join them into the prompt with the instructions, generate.

▶ 2:15 Update a document and the next answer uses it, with no retraining (illustrative).

RAG is how most assistants answer from **private or fresh** information **without retraining** the model: update a document, and the next answer uses it. But everything depends on **retrieving the right pieces**, which is harder than it looks (next episode).

KEY IDEA

To change what a RAG system knows, change the library, not the model.

Check 4: Why RAG is everywhere

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 TRUE OR FALSE

"To teach a RAG assistant a new opening time, you must fine-tune the model."

True ☐ False ☐

4.2 HANDS-ON CODE

Try it yourself. In `code/p05_rag/rag.py`, set `QUESTION` to each question below and run it. (a) Which two facts are retrieved for each, and what does the 1.5B model answer? (b) The second answer is not wrong, but it is unhelpful. Why, and how could you improve it?

```
QUESTION = "Is Bella's Bakery open on Monday at 8:00?"
QUESTION = "Do you deliver 3 km away for a 20 euro order?"
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The gap RAG fills · rewatch ▶ 0:08-0:32

1.1 B. Only B is common knowledge likely to be in the training data.

1.2 False. Inventing a plausible but false answer is worse: it can mislead you.

2. Retrieve, then generate · rewatch ▶ 0:32-1:20

2.1 embed the library (once) → embed the question → take the top-k matches → build the prompt → generate the answer. Library vectors are computed ahead of time; the rest happens per question.

2.2 To stop the model from guessing when retrieval did not find the answer. Without it, the model may fill the gap with something invented.

2.3 0.60. The top three are 0.70, 0.62 and 0.60.

3. Right fact, wrong answer · rewatch ▶ 1:20-1:54

3.1 B. The top retrieved fact was exactly the right one (0.70).

3.2 For example: ask the model to quote the source sentence it used and check it; use a more capable model; test on questions with known answers. A quoted source makes a contradiction like “only on Fridays” → “yes on Wednesday” easy to spot.

4. Why RAG is everywhere · rewatch ▶ 1:54-2:23

4.1 False. Update the document in the library; the next retrieval returns the new text.

4.2 (a) Monday: the weekday hours (0.88) and “closed on Mondays” (0.82); answer: “No, Bella's Bakery is closed on Mondays.” Delivery: the delivery rule (0.80) and the sourdough price (0.28); answer: “I don't know...”, although it quotes the rule. (b) The answer follows from the rule (20 euros is under the 30-euro minimum, so no), but the model did not apply it. Fixes: a more capable model, or an instruction such as “apply the rules to the question and explain your reasoning”. Retrieval was right both times; the difference is in how well the model reasons over the text.